NTA Data Analysis

Full Technical Specification

CSC1049 – Year 3 Project

Kevin Biju – 22332903

Daragh Montague – 22375933

1. System Overview

1.1 Purpose

NTA Data Analysis (Jump) is a full-stack data engineering and machine learning project that analyses how reliable Ireland's public transport network actually is. We use GTFS static schedules and GTFS-Realtime feeds from the National Transport Authority to build up a picture of delay patterns across the network.

On the pipeline side, we ingest static timetables, poll live TripUpdates continuously, store raw snapshots for auditability, construct delay events by joining predicted and scheduled times, and train a predictive model. The frontend is a React dashboard with a journey planner that ranks routes by reliability instead of just travel time.

What the project covers:

- End-to-end data pipeline engineering
- PostgreSQL schema design with separate schemas for static, realtime, and analytics data
- Ingestion with full data lineage tracking
- Delay event construction from transport datasets
- Time-aware XGBoost modelling for delay prediction
- Automated polling, analytics building, and model retraining
- Full-stack web dashboard with Google Maps, journey search, and reliability visualisation
- REST API with geospatial queries and temporal filtering

1.2 Division of Work

Kevin built the frontend dashboard, the backend REST API, the route search algorithms, reliability visualisations, and the overall UX. Daragh built the data ingestion pipelines, the GTFS static and realtime processing, analytics construction, and the ML model. We shared work on the database schema, integration, and testing.

2. High-Level Architecture

2.1 Technology Stack

There are four main layers: database, backend API, data pipelines, and frontend.

Database: PostgreSQL 16 in a Docker container on port 5433. Schema changes are managed through Alembic migrations.

Backend: Python 3.12 with FastAPI, serving a REST API on port 8000. We use SQLAlchemy for DB access but drop down to raw SQL for anything performance-sensitive.

Frontend: React 18 with TypeScript, built with Vite 5. Tailwind CSS 3.4 for styling, Google Maps API for the maps, React Router for client-side navigation.

ML: Scikit-learn and XGBoost for the delay prediction model. Pandas and NumPy for data wrangling.

Data Sources: NTA GTFS-Realtime API for live vehicle updates, NTA GTFS static dataset for timetable reference.

2.2 System Flow

Static GTFS data gets downloaded, validated, and loaded into the gtfs_static schema. GTFS-Realtime TripUpdates are polled on a loop and stored as raw snapshots in gtfs_rt. A delay builder joins the realtime predictions with scheduled times to produce delay events in the analytics schema. Those events feed into model training.

The API serves all this to the React dashboard, which shows it through maps, reliability gauges, and the journey planner. An orchestration script ties it together: continuous polling, analytics rebuilds every five minutes, model retraining every three hours.

3. Database Design (PostgreSQL)

Everything runs on PostgreSQL with Alembic for migration management.

3.1 Schemas

- gtfs_static – timetable reference data (routes, stops, trips, stop_times, etc.)
- gtfs_rt – raw realtime snapshots
- analytics – derived delay events and model training metrics
- gtfs_static_staging – temp schema used during atomic static data loads

3.2 Static GTFS Schema

The core tables are agency, routes, stops, calendar, calendar_dates, trips, stop_times, and shapes. Foreign keys enforce referential integrity and we've added indexes on the columns used

in joins and lookups. stop_times is the most important table for journey planning since it stores arrival/departure times with stop_sequence for ordered traversal.

3.3 Realtime Storage

The feed_snapshots table stores the raw protobuf payload along with the HTTP status, ETag, SHA256 hash, entity count, and header timestamp. Keeping the raw data means we can always reparse and reprocess if our analytics logic changes.

3.4 Analytics Schema

trip_delay_events is where the processed delay data lives. Each row records the route, stop, trip, predicted arrival time, scheduled arrival time, delay in seconds, and prediction lead time. There's also a model_training_metrics table that logs evaluation results from each training run: MAE, median error, and percentage within various thresholds.

4. Data Pipelines

4.1 Static GTFS Pipeline

Static datasets get downloaded from a configured URL, validated to check they have all the required GTFS files and dates are formatted properly, then loaded into a staging schema. Once everything looks good, it's merged into the production gtfs_static schema atomically. This prevents half-loaded data from messing things up.

4.2 Realtime Polling Pipeline

A drift-free polling loop hits the NTA GTFS-Realtime TripUpdates endpoint at a configurable interval (default 30 seconds, set via GTFS_RT_POLL_SECONDS environment variable). It uses ETags for conditional requests so we don't re-download identical data, and hashes each payload with SHA256 for deduplication.

4.3 Delay Event Construction

Each realtime snapshot gets parsed and joined to the static schedule data to produce delay events. We use timestamp-based tracking (fetched_at column) rather than snapshot IDs because we import data from different machines and the IDs don't always line up. Each event includes predicted time, scheduled time, delay in seconds, and how far ahead the prediction was made.

4.4 Pipeline Orchestration

run_pipeline.py is the main entry point that manages everything: continuous RT polling in a background thread, analytics rebuilds every five minutes, and model retraining every three hours. The pipeline status gets exposed through the API overview endpoint so the dashboard can show whether it's running or not.

5. Machine Learning System

The prediction model is trained on delay events from the last 7 days.

Features: route_id (one-hot encoded), stop_sequence, prediction_lead_time_seconds, hour, and day_of_week. Target variable is delay_seconds.

We do a time-based train/test split to avoid temporal leakage (training on older dates, testing on newer ones). The model is an XGBoost regressor.

Each training run logs:

- Mean Absolute Error (MAE)
- Median Absolute Error
- Percentage of predictions within 60 seconds
- Percentage of predictions within 120 seconds

6. Backend API

The backend runs FastAPI on port 8000 with CORS configured to allow requests from the dev frontend. It's split into five router modules: routes, stops, trips, search, and analytics.

6.1 Routes Endpoints

GET /api/routes returns a paginated route list with optional filtering by route_type (bus, rail, tram, etc.) and agency_id. GET /api/routes/{route_id} gives you the route details plus all stops it serves in sequence order. GET /api/routes/{route_id}/trips lists trips on the route with direction filtering.

6.2 Stops Endpoints

GET /api/stops does full-text search on stop names with ILIKE and also supports geospatial proximity search using Haversine in SQL. If you pass coordinates and a radius, it returns stops within range with their distance in km. GET /api/stops/{stop_id} returns the stop details and all routes serving it. GET /api/stops/{stop_id}/departures gives the departure schedule with trip info.

6.3 Trips Endpoints

GET /api/trips/{trip_id} returns the trip with its route, stop sequence, arrival/departure times, and coordinates. GET /api/trips/{trip_id}/shape returns the polyline geometry for drawing the route on the map.

6.4 Search Endpoints

This is where the journey planning logic lives.

GET /api/search/routes finds direct routes between two points. It identifies stops near the origin and destination using Haversine, then checks for routes that serve both areas by looking at trip

continuity through stop_sequence ordering. Results are scored using travel time and walking distance (walk distance gets a 15x penalty factor) and sorted by reliability first, then walking distance.

GET /api/search/journeys adds transfer support. For transfers, it finds hub stops where two routes overlap, filters by geographic bounds around the journey midpoint, and estimates duration from Haversine distance and walking speed. Transfer journeys get deduplicated by route pair and transfer stop. Rail and Luas get priority in sorting. Combined reliability for multi-leg journeys is calculated by multiplying the individual leg reliabilities.

GET /api/search/stops-nearby returns stops within a configurable radius, ordered by distance.

6.5 Analytics Endpoints

GET /api/analytics/route/{route_id}/reliability returns on-time percentage, average delay, median delay, and sample size. You can pass day_of_week and hour parameters to filter by time period. On-time is defined as within 300 seconds (5 minutes) of schedule.

GET /api/analytics/stop/{stop_id}/reliability gives per-route reliability at a specific stop so you can see how each route performs at that location.

GET /api/analytics/overview returns system-wide stats: counts of routes, stops, trips, and agencies from static data, plus realtime metrics like snapshot count, delay event count, latest event timestamp, and recent pipeline run history.

7. Frontend Dashboard

The dashboard is a React + TypeScript SPA built with Vite. We use Tailwind CSS with a glassmorphic design language (frosted glass cards, backdrop blur, translucent backgrounds). Dark mode is supported with system preference detection and a manual toggle. React Router handles client-side navigation.

7.1 Application Structure

The frontend is organised into pages, components, context providers, and an API client layer. Pages handle the main views and data fetching. Components are the reusable UI pieces. Context providers manage theming and favourites state. The API client gives us type-safe functions for every backend endpoint.

7.2 API Client

The client module has TypeScript interfaces for all data types (Route, Stop, Journey, JourneyLeg, SystemOverview, StopReliability, etc.) and typed fetch functions for every endpoint. There are also utility functions that map GTFS route types (0–7) to readable names and emoji icons for visual differentiation.

7.3 Home Page

The home page is the main landing screen. It has a hero section linking to the journey planner, a favourites section for saved routes, a Google Map showing transport stops around Dublin, and a stats grid with route/stop/trip/event counts. There's also a pipeline status card showing if the realtime collection is running, snapshot and event counts, when it last updated, and recent ingestion runs with colour-coded status badges.

Transport operators show up as gradient cards grouped by type. The big ones (Dublin Bus, Go-Ahead, Bus Eireann, Irish Rail, Luas, DART) are displayed prominently with icons. Local link and smaller services are in a collapsible section. Popular routes show in a carousel with favourite toggle buttons.

7.4 Journey Search Page

This is the main feature page. Users type origin and destination into autocomplete inputs that search stops by name with debounced input. The dropdown shows matching stops and supports keyboard navigation (arrow keys, Enter, Escape). You can also click the map to pick a location, and there's a visual indicator showing which field is in map-selection mode.

Results appear as journey cards showing direct vs transfer, estimated duration, per-leg reliability gauges, stop names with walking distances, and transfer info with dashed connector lines. Each card links through to a full trip summary.

7.5 Route Details Page

Shows a header with route number, name, type badge, and operator. There's a favourite button to save it. The reliability section has a large circular gauge with on-time percentage, average delay in minutes, and sample size. Dropdown filters let you pick a specific day of week and hour to see how reliability changes at different times. Below that is a map with all the stops and a scrollable stop list in sequence order.

7.6 Stop Details Page

Shows stop name, ID, stop code, and coordinates. The map centres on the stop with a link to open it in Google Maps. All routes serving that stop are listed with individual reliability gauges per route, so you can see which routes are reliable at that particular stop.

7.7 Trip Summary Page

This page gives a detailed breakdown of a selected journey. The header shows direct/transfer badge, total duration, the origin-to-destination path, and an overall reliability gauge. A map shows origin and destination markers plus any transfer points.

Each leg gets its own card with route info, an SVG gradient timeline, number of intermediate stops, walking distances, and per-leg reliability stats. For transfer journeys there's a combined section showing overall on-time probability (multiplied across legs) and total average delay.

7.8 Reusable Components

GlassCard is a wrapper that gives things the frosted glass look with backdrop blur, translucent backgrounds, and an optional hover lift. It comes in three variants (default, subtle, strong) and is keyboard-accessible.

ReliabilityGauge is a custom SVG circular progress indicator. It colour-codes dynamically: green above 85%, amber for 70–84%, red below 70%, grey when there's no data. Three sizes (small, medium, large) for different layout contexts.

GoogleMap wraps the Google Maps JS API. It handles multiple marker types with colour coding (origin green, destination red, stops indigo), info windows for metadata, polyline route rendering, dark mode map styling, auto-zoom to fit markers, and click handling for the search location picker.

7.9 State Management

We use React Context for global state. ThemeContext handles dark/light mode with localStorage persistence and system preference detection. FavoritesContext manages saved routes, also persisted to localStorage, with add/remove/toggle/check operations and timestamps.

8. Non-Functional Characteristics

Performance: The backend uses indexed queries and Haversine filtering in SQL for fast API responses. The frontend does parallel data fetching with Promise.all() and shows loading skeletons so things feel responsive.

Reliability: We keep all raw snapshots so we can always reprocess. The pipeline tracks by timestamp rather than ID to avoid conflicts when importing data from different environments.

Security: API keys live in environment variables, not in the repo. CORS restricts API access to our frontend origin.

Usability: Dark mode, keyboard nav, responsive layouts, and useful empty states throughout the app.

Maintainability: Modular architecture on both frontend (pages/components/context/API) and backend (separate routers). Alembic for DB migrations.

9. Testing

We have unit and integration tests in pytest. Unit tests cover data validation and transformation functions. Integration tests check the RT polling and static pipeline against the actual database. Tests live under code/tests/ split into unit/ and integration/ directories.